TRƯỜNG ĐẠI HỌC FPT

**CAPSTONE PROJECT REGISTER**

**Class**: **Duration time**: from 09/2026 To 03/2027

**(*) Profession:** Software Engineer **Specialty**: SE [x]

**(*) Kinds of person make registers:** Lecturer [x] Students

**1. Register information for supervisor (if have)**

| No. | Full name | Phone | E-Mail | Title |
|---|---|---|---|---|
| Supervisor | Đặng Ngọc Minh Đức | | ducdnm2@fe.edu.vn | Giảng viên |
| Co-supervisor | Bùi Văn Phúc | | | Giảng viên |

**2. Register information for students (if have)**

| No. | Full name | Student code | Phone | E-mail | Role in Group |
|---|---|---|---|---|---|
| 1 | Đỗ Đăng Khoa | SE192357 | | khoado7577@gmail.com | Leader |
| 2 | Trần Khải Hoàng | SE182767 | | HoangTKSE182767@fpt.edu.vn | Member |
| 3 | Lâm Phi Long | SE182769 | | LongLPSE182769@fpt.edu.vn | Member |
| 4 | Nguyễn Bá Tân | SE183860 | | TanNBSE183860@fpt.edu.vn | Member |
| 5 | Nguyễn Ngọc Long | SE193490 | | nguyenngoclong216@gmail.com | Member |

**3. Register content of Capstone Project**

**(*) 3.1. Capstone Project name:**

3.1.1. English: ***TrekLink: An Integrated Off-Grid Communication and Smart Device Rental Management Platform for Trekking Agencies***

3.1.2. Vietnamese: ***TrekLink: Nền tảng tích hợp liên lạc ngoài vùng phủ sóng và quản lý cho thuê thiết bị thông minh cho doanh nghiệp trekking***

**Abbreviation: TrekLink**

**(*) 3.2. Main proposal content (including result and product)**

**a) Context:**

**Market context.** Trekking in Vietnam often takes place in remote mountainous areas, on routes such as Tà Năng–Phan Dũng, Tả Liên Sơn and Bạch Mộc Lương Tử, where cellular

connectivity is limited or unavailable for long stretches. Many groups from different companies share the same routes on the same day.

**Identified gap.** Integrated platforms combining off-grid IoT communication, monitoring, device management and structured incident coordination are not yet widely adopted in Vietnam's trekking service market.

**Project opportunity.** TrekLink addresses this gap by extending its existing LoRa mesh communication technology into an integrated operational platform designed for the Vietnamese trekking environment.

During Summer 2026, the project team developed TrekLink, an open-source LoRa mesh communication firmware for ESP32/ESP32-S3 devices. The firmware, authored by this team and based on Meshtastic, supports multi-hop mesh messaging, GPS tracking, fall detection, and automatic SOS broadcasting. It is version-controlled and serves as the inherited technical foundation for this Capstone; it will not be rebuilt.

TrekLink is a smart-device management platform whose customers are trekking companies; it does not manage tours. TrekLink owns the devices, the firmware, the Field Station software and the cloud platform, and rents devices to organizations by plan. Many organizations use one platform, each managing its own members, on-duty roster, rented devices, alerts and API keys; each keeps its own trips, customers and emergency response outside the platform.

**Scope Boundary:** The LoRa mesh firmware and the hardware built by this team in Summer 2026 are the technical foundation. Targeted firmware enhancements made during the Capstone (the durable on-device queue, SOS beaconing fixes, radio power compliance) are in scope and credited; rearchitecting the mesh stack is not. The Capstone covers the Field Station (gateway bridge), offline-resilient synchronization, the cloud backend, organization and asset management, rental contracts, incident management with tiered alerts, live telemetry with an organization API, billing, system integration, and experimental evaluation.

The needs this Capstone serves, per party:

- **Trekkers and guides:** share position and raise an alert from places a phone cannot reach.
- **Trekking companies:** see their own groups and devices, and receive alerts with a clear owner and a record, without buying or maintaining the hardware.
- **Local authorities and rescue units:** receive a timely report with the last known position and the history of signals when an incident escalates.
- **TrekLink, as the provider:** track every rented device's custody, condition and charges across many renters; deliver field events through long outages.

**b) Proposed Solutions:**

TrekLink extends the existing LoRa mesh firmware into an enterprise device-rental platform built on asset management through six core capabilities:

- **Organization and rental management:** Organizations self-register and are verified and approved by TrekLink. They rent devices by monthly plan (half the term fee at the start as a holding fee, half at the end, renewing until notice) or by day plan (3, 4 or 7 days, paid in full at handover, priced above the monthly rate per day), with a minimum order quantity. Handover and return happen in person at the TrekLink counter with a signed handover note; payments are recorded, online through a third-party payment gateway (VietQR, sandbox) or at the counter.
- **Asset management:** Each physical device has two identities, a printed asset tag and its radio node number, and a nine-state lifecycle: In Intake, Available, Reserved, Rented, Returned, Maintenance, Lost, Retired and Disposed. The main transitions are: Available to Reserved at contract approval, Reserved to Rented at handover, Rented to Returned at check-in, Returned to Available after reset and a passed inspection or to Maintenance after a failed one, Maintenance to Available after repair or to Retired, and Retired to Disposed after a verified wipe. A loss can be reported from any of the six working states; it opens a loss case, which ends with the device recovered to Returned or with the loss settled and the device Retired. The renter bears a loss only when the device was in its custody. Devices are provisioned with the renter's channel key at handover and reset and inspected on return before they can be rented again.
- **Offline-Resilient Field Station:** Each field event is identified by eventId = sha256(nodeNum : packetId), derived from identifiers the firmware already transmits. When Internet connectivity is unavailable, the Field Station, a single executable that the organization runs on its own laptop with one rented node on a USB port, persistently stores events in a local SQLite queue and shows the local mesh to the operators on site. Safety-critical SOS events are assigned P0 priority and synchronized first on reconnection; GPS updates are P2 and routine telemetry P3. Each device also keeps its own durable queue. Each event records when it occurred and the source of that time, and belongs to the organization that held the device at that moment, so an event that synchronizes after the device was returned and rented again reaches the earlier renter and never the new one.
- **Idempotent Event Processing:** The cloud backend uses the Event ID as an idempotency key. Regardless of how many times the same event is delivered, due to retransmission, reconnection replay, or duplicate LoRa broadcast, the backend creates exactly one Incident and one alert per tier, preventing alert fatigue and ensuring data consistency.
- **Incident Management with tiered alerts:** Each SOS episode becomes one incident routed to the organization holding the device. Alerts go to its on-duty member, then its backup, then its Manager, each after a timeout; if nobody acknowledges, the incident escalates to TrekLink, which reports to the authorities. The organization runs the response and reports status back. Every transition is atomic and audited with actor, time and reason.
- **Live telemetry:** Each organization sees only its own rented devices, positions, battery levels and alerts on a live map, and can read the same data in its own systems through a REST and WebSocket API with an organization key.

Technology stack:

- **Embedded firmware:** ESP32/ESP32-S3 + TrekLink firmware (team-built; targeted enhancements in scope)
- **Field Station:** Node.js (TypeScript) packaged as a single executable with a local web page: async I/O for concurrent MQTT + LoRa serial communication
- **Backend:** NestJS (TypeScript): modular architecture with REST and WebSocket support
- **Database:** PostgreSQL with Prisma ORM: ACID compliance for rental transactions and incident audit trail
- **IoT messaging:** MQTT (Mosquitto): QoS-based delivery between Field Station and backend
- **Real-time push:** Socket.io WebSocket: live dashboard updates and the organization API stream
- **Frontend:** React + TypeScript: role-based views with Leaflet.js map integration
- **Payment gateway:** third-party payment gateway (VietQR with confirming webhook), sandbox environment
- **DevOps:** Docker Compose + GitHub Actions: containerised deployment and CI/CD

**c) Functional Requirements:**

- **TrekLink Admin**:

• Approve or reject organizations, and suspend or reactivate them.

• Manage platform accounts, roles, permissions, and system configurations.

• Manage TrekLink hardware variants, firmware versions, and their remaining-value schedules.

• Manage rental plans and prices: monthly price per variant, day-plan lengths and premium, minimum order quantity, late, damage and loss charges.

• Approve damage charges above the threshold (never as the inspector) and retire devices.

• Monitor system health, Field Station connectivity, alert delivery, and incidents; view audit logs and management reports.

- **TrekLink Staff**:

• Verify organizations in person or by phone and record the signed master contract, including the non-disclosure agreement and the Field Station software licence; record suspected fraud, theft, IP or NDA breaches as compliance cases.

• Register devices (asset tag and node number), run intake checks, and record maintenance.

• Approve rental contracts, reserving devices without over-allocation.

• Provision devices with the organization's channel key and hand them over at the counter with a signed handover note.

• Check devices in, reset and inspect them, record damage and loss, and close contracts.

• Record counter payments, and run stock-takes.

• Handle escalated and unrouted incidents and record reports to the authorities.

- **Organization Manager:**

• Register the organization, sign the master contract and handover notes.

• Request rental contracts (plan, hardware variant, quantity, start date), pay online or at the counter, and give notice on monthly contracts.

• Manage the organization's members, on-duty roster (primary and backup per shift), and API keys.

• View contracts, terms, invoices, and payments; act as the last organization alert tier.

- **Organization Operator (on-duty member):**

• Receive SOS alerts, acknowledge incidents, report response status and outcome, or declare a false alarm.

• View the organization's rented devices, positions, battery, and alerts on the live map.

• Run the Field Station on site and view the local mesh offline; optionally label each device with its holder.

- **Organization System (API client):**

• Read the organization's live positions, telemetry, and incident events over REST and WebSocket with an organization API key.

- **Asset Management:**

• Register each physical unit with asset tag, node number, hardware variant, firmware version, and status; neither identity is ever reused.

• Manage the nine-state device lifecycle (In Intake, Available, Reserved, Rented, Returned, Maintenance, Lost, Retired, Disposed) through its allowed transitions only; for example, a returned device goes back to Available after a passed inspection, and a device in any working state can be reported Lost.

• Handle each lost device as a loss case: custody decided from the rental history, channel key exposure flagged, recovery or a charge by custody with an Admin waiver, and a police report reference for suspected theft or fraud; replace a faulty or lost unit on a running contract at the counter.

• Record battery level, last-seen time, latest GPS position, and telemetry status.

• Maintain custody, contract, incident, inspection, and maintenance histories; reset every returned device before it can be rented again.

• Schedule maintenance by a service plan per hardware variant and a weekly scan, record its cost with Admin approval above a threshold and each firmware update, and prevent unavailable or due-for-service devices from being allocated.

- **Event Reliability & Offline Synchronization:**

• Each field event shall contain an eventId derived as sha256(nodeNum : packetId), source device ID, event type, UTC timestamp, and priority tier.

• The Field Station shall persist unsent events in a local SQLite queue when cloud connectivity is unavailable, and each device shall keep its own durable queue.

• SOS and safety-critical events (P0) shall receive higher synchronization priority than GPS updates (P2) and routine telemetry (P3). On reconnection, P0 events shall be delivered before any lower-priority queued events, regardless of their original arrival order.

• The Gateway shall automatically flush queued events after connectivity is restored, reporting synchronization status, retry count, and queue depth per priority tier.

• The backend shall process all incoming events idempotently using eventId as the idempotency key.

• Duplicate reception or retransmission of the same SOS event shall create exactly one Incident and one logical notification event in the backend.

• The system shall record synchronization status, retry count, and processing history for each event for diagnostic and audit purposes.

• Acceptance criterion: repeated delivery of the same SOS event (identical eventId) shall produce exactly one Incident record and one notification, regardless of delivery count.

- **Live Telemetry & Incident Management:**

• Display each organization’s rented devices, positions, device status, and last-seen information on a real-time map, scoped to that organization on the server; offer the same data through the organization API.

• Create an Incident automatically when a valid SOS event is received and idempotency check passes, and route it to the organization holding the device.

• Alert the organization’s on-duty member within 2 seconds of incident creation, then the backup and the Manager after configurable timeouts; escalate to TrekLink when nobody acknowledges.

• Support the incident lifecycle with tiered notification, acknowledgement, response, resolution, false alarm, escalation, authority report, reopen, and close.

• Record actor (user ID + role), timestamp, and action note at each state transition.

• Preserve a complete, append-only incident history for audit purposes.

• Keep an incident owned at every moment: either a named owner or a running tier timeout.

- **Billing & Reporting:**

• Calculate monthly term fees (holding fee and balance) and day-plan fees per device and hardware variant, plus late, damage, and loss charges.

• Generate invoices and track payment status.

• Integrate a third-party payment gateway (VietQR, confirming webhook) in its sandbox environment, and record counter payments.

• Generate reports on contracts, fleet utilization, revenue, overdue and lost devices, maintenance, incidents, and response performance.

**d) Non-Functional Requirements:**

- **API Performance:** At least 95% of normal backend API requests shall complete within 300 ms under the defined test workload of 50 concurrent users.
- **Synchronization Latency:** When gateway uplink connectivity is available, newly received field events shall reach the cloud backend within 5 seconds under normal load conditions.
- **Offline Recovery: Delivery Rate:** The Gateway shall achieve a message delivery rate of ≥ 99% after connectivity recovery under the defined experimental conditions (connectivity-loss durations of 30 s to 30 min, minimum 20 trials per condition).
- **Offline Recovery: Testability:** Connectivity loss shall be reproducible in a controlled lab environment by programmatically interrupting the gateway's network interface, allowing repeatable experiment execution.
- **Duplicate Prevention:** Zero duplicate Incident records shall be created for repeated delivery of any SOS event carrying the same eventId. This shall be verified by a dedicated automated test that delivers the same SOS event 10 times and asserts exactly one Incident is created.
- **Priority Ordering:** On every reconnection, all P0 (SOS) events queued during the offline period shall be delivered to the backend before any P2 or P3 events. Priority-ordering compliance rate shall be ≥ 99% across all experimental trials.
- **Concurrency:** The backend shall process at least 20 simultaneous gateway event submissions without message loss or duplicate Incident creation, verified by a concurrent load test.
- **Offline Resilience:** Loss of cloud connectivity shall not interrupt local LoRa mesh communication or device-level SOS propagation on the field devices.
- **Data Consistency:** Replayed gateway messages shall not create duplicate incidents or duplicate telemetry records, enforced by eventId-keyed idempotency at the backend.
- **Security:** JWT-based authentication, RBAC, TLS-encrypted transport, bcrypt password hashing, and auditable access to sensitive incident data. No organization shall be able to read another organization's devices, positions, incidents, contracts, or invoices, verified by a cross-organization test per module.
- **Auditability:** Every incident state transition, device lifecycle transition, and rental contract event shall be traceable to an actor (user ID + role) and a UTC timestamp.

- **Scalability:** The system shall support at least 10 organizations and 50 rented TrekLink devices concurrently without architectural changes.
- **Deployability:** All server components shall support containerized deployment via Docker Compose and automated CI/CD via GitHub Actions.

**e) Theory & Practical:**

**Theory: The project applies software engineering and distributed systems principles to integrate an intermittently connected LoRa mesh network with a cloud-based operational platform.**

- **Finite State Machines:** Model the nine-state device lifecycle with its loss case, the incident lifecycle with tiered notification and escalation, and the organization and rental-contract lifecycles with controlled, auditable state transitions. Each transition is an atomic, logged operation.
- **Distributed Systems and Eventual Consistency:** The Gateway and cloud backend are loosely coupled through MQTT. Field events are produced offline and delivered when connectivity permits; the backend converges to a consistent state after all queued events are processed.
- **Priority-Aware Store-and-Forward:** Classify field events by safety criticality (P0 SOS → P1 incident-related location → P2 GPS tracking → P3 routine telemetry) and enforce priority ordering during queue flush, ensuring safety-critical events are not delayed behind routine data.
- **Idempotent Event Processing:** Backend event handlers produce the same outcome regardless of delivery count, using eventId as a natural idempotency key. This decouples reliability from the transport layer and eliminates the need for exactly-once delivery guarantees at the MQTT level.
- **IoT Messaging Protocols:** MQTT QoS 1 (at-least-once) is used in conjunction with backend-side idempotency to achieve effective exactly-once processing without requiring QoS 2.
- **Event-Driven Architecture:** SOS events trigger an asynchronous processing pipeline: event ingestion → idempotency check → incident creation → role-based notification → audit logging, where each stage is decoupled and independently testable.
- **Role-Based Access Control (RBAC) and multi-tenancy:** Fine-grained permissions enforced at the API layer for TrekLink Admin and Staff and for each organization's Manager and Operators, with every organization's data isolated on the server.
- **Software Architecture:** Domain-driven modular design separates gateway synchronization, organizations, devices, rental contracts, incidents, monitoring, billing, and platform services into independent NestJS modules with documented REST and WebSocket APIs.

**Practical: Build on the team's ESP32/ESP32-S3 LoRa mesh firmware and hardware (Summer 2026), with targeted enhancements. Document the complete message schema (eventId structure, message types, payload format) before any gateway implementation begins.**

- Develop the Field Station (Node.js/TypeScript): LoRa-to-serial message parser, SQLite persistent priority queue, MQTT uplink publisher, reconnection flush logic, Field Station health reporter, local web page, and single-executable packaging.
- Develop the NestJS backend: auth/RBAC module, organizations module, asset (device) module, rental contract module, gateway sync module (idempotent event ingestion), incident module (tiered alerts + WebSocket notifications), billing module (payment gateway), and reporting module. Apply PostgreSQL migrations via Prisma.
- Develop the React/TypeScript web client: role-based views for TrekLink and for organizations, real-time operational map (Leaflet.js), incident dashboard with WebSocket updates, contract and asset management interfaces, the organization API, and reporting views.
- Deploy with Docker Compose; automate testing and deployment with GitHub Actions CI/CD.
- Design and execute the experimental protocol for RQ1, RQ2, and RQ3 as specified in the Research Description section.

**f) Products (Expected Deliverables):**

- **TrekLink Rental Platform:** Integrated web application for TrekLink Admin and Staff and for each organization's Manager and Operators.
- **Field Station:** LoRa-to-cloud gateway run on the organization's laptop with persistent SQLite priority queue (P0–P3), idempotent MQTT synchronization, reconnection flush, health reporting, and a local offline view. Primary new hardware-integration deliverable.
- **Event Reliability Layer:** End-to-end eventId scheme, backend idempotency enforcement, synchronization audit log, and duplicate-prevention test suite.
- **Organization & Rental Contract Module:** Organization registration and approval, members, on-duty roster, API keys, monthly and day plans, handover, terms, notice, overdue and default.
- **Asset Management Module:** Nine-state device lifecycle, intake, provisioning, reset, inspection, scheduled maintenance with cost approval, loss cases, replacement, disposal, and stock-take.
- **SOS Incident Management Module:** Automatic Incident creation from SOS events, tiered alerts with escalation to TrekLink and authority-report records, WebSocket notification, response tracking, and append-only audit history.
- **Live Telemetry Module:** Organization-scoped live map, device telemetry, positions, Field Station connectivity, and a REST and WebSocket API per organization.
- **Billing Module:** Term and day-plan pricing, late, damage and loss charges, invoices, and online payments through the payment gateway (sandbox).
- **Backend API & Real-Time Services:** REST API (OpenAPI/Swagger documented) and Socket.io WebSocket services.
- **Technical Documentation:** SRS with UML State Machine Diagrams (Device, Incident, Organization, Rental Contract), system architecture, database schema, API reference, test plan, deployment guide, and user guide.

- **Evaluation Report:** Experimental results for RQ1 (gateway delivery rate, latency, duplicate rate), RQ2 (effect of connectivity-loss duration), and RQ3 (SOS-to-Incident workflow vs. baseline).
- **Source Code & Deployment Package:** Version-controlled repositories (gateway, backend, frontend), Docker Compose configuration, GitHub Actions CI/CD, and a runnable demonstration environment.

**g) Proposed Tasks:**

- **TP1: Requirements, Architecture & Experimental Design (Weeks 1–3):** SRS, UML State Machine Diagrams (Device Lifecycle, Incident Lifecycle), system architecture document, inherited-component analysis (firmware message schema, eventId structure), database schema design, RQ/experiment protocol design, and proof-of-concept LoRa-to-Gateway serial message parser. All subsequent TPs depend on the frozen message schema produced in TP1.
- **TP2: Gateway Bridge & Offline Synchronization (Weeks 2–6):** LoRa-to-Gateway serial integration, SQLite persistent priority queue with P0–P3 tiers, MQTT publish logic, reconnection detection and flush routine, duplicate-prevention via eventId at gateway level, gateway health monitoring API, and unit/integration tests covering offline queue, priority ordering, and reconnection recovery.
- **TP3: Core Backend & Reliability Services (Weeks 3–7):** NestJS authentication and RBAC module, device fleet FSM module, rental lifecycle module, idempotent event ingestion endpoint (eventId-keyed deduplication), synchronization audit log, PostgreSQL schema and migrations, and automated tests for idempotency (10× same eventId → 1 Incident) and concurrent event submission (20 simultaneous requests → 0 duplicates).
- **TP4: Monitoring & Incident Management (Weeks 5–9):** Real-time telemetry ingestion, Leaflet.js organization-scoped map with WebSocket live updates, the organization telemetry API, SOS-to-Incident processing pipeline, tiered-alert incident FSM with escalation, acknowledgment workflows, and append-only incident audit trail.
- **TP5: Business Operations (Weeks 6–10):** Organization onboarding, rental contracts and plans, device reservation, handover and check-in at the counter, reset and inspection, billing with online payments, TrekLink and organization web views, operational dashboards, audit log views, and management reports.
- **TP6: Integration, Experimental Evaluation & Deployment (Weeks 9–13):** Physical LoRa experiment execution (RQ1/RQ2: 3–5 devices, connectivity-loss matrix, ≥ 20 trials per condition), offline/recovery experiment results, duplicate-event test results, concurrency/load test results, cross-organization isolation suite results with at least two organizations per module, simulated multi-organization load test results, RQ3 SOS drill evaluation (randomized, ≥ 15 drills per condition, automatic timestamp recording), evaluation report, end-to-end integration test suite, Docker Compose deployment, GitHub Actions CI/CD pipeline, and final Capstone documentation.

Risk assessment:

- **Insufficient physical TrekLink devices** (Likelihood: Medium / Impact: High): Simulate additional Gateway-to-Cloud load using MQTT scripts. Physical devices are required

only for mesh-to-gateway experiments; simulation shall not be used to claim LoRa RF reliability.

- **Gateway sync latency exceeds 5-second NFR** (Likelihood: Medium / Impact: Medium): Tune MQTT QoS level, reduce payload size, implement backpressure on the queue flush rate.
- **RQ3 baseline not objectively measurable** (Likelihood: High / Impact: Medium): Use randomized simulation drills; independent observer triggers SOS and records timestamp; participants do not know exact trigger time in advance.
- **NestJS + MQTT + WebSocket integration underestimated** (Likelihood: Medium / Impact: High): Proof-of-concept gateway-to-backend integration completed in TP1 Week 2; does not wait for TP3.
- **Firmware message schema incompatible with new gateway** (Likelihood: Low / Impact: High): Schema frozen and documented in TP1 before any gateway implementation begins.
- **Scope creep** (Likelihood: High / Impact: Medium): Feature freeze after TP5 Week 10; additional features go to post-Capstone backlog.

### 3.3. Research Information

#### a. Research Problem / Research Question

Trekking companies and similar organizations operating where cellular coverage is limited need field events, SOS alerts included, to reach their staff even after a long outage, and need each SOS handled as a coordinated incident with an owner, tracked acknowledgment and an audit trail.

The Summer 2026 TrekLink firmware provides the field communication layer. This Capstone adds a Field Station with priority-aware offline queuing and idempotent event processing, and an SOS-to-Incident pipeline with a formally defined lifecycle and tiered escalation, and evaluates both.

Three research questions operationalize the evaluation:

- **RQ1:** How reliably can the TrekLink Gateway Bridge synchronize SOS, GPS, and telemetry events from an intermittently connected LoRa mesh to the cloud backend, measured by message delivery rate, data-loss rate, and duplicate rate?
- **RQ2:** How do varying connectivity-loss durations and reconnection patterns affect message delivery rate, synchronization latency, and priority-ordering compliance of the proposed Gateway architecture?
- **RQ3:** Compared with the existing uncoordinated device-level SOS workflow, to what extent does the TrekLink SOS-to-Incident pipeline improve incident response coordination, traceability, and acknowledgment consistency, as measured by MTTA, MTTR, traceability score, and incident-completion rate in controlled emergency drills?

RQ1 and RQ2 directly evaluate the Event Reliability & Offline Synchronization functional requirements. RQ3 directly evaluates the Incident Management functional requirements and the

incident FSM. All three RQs trace back to specific functional requirements, system modules, task packages, and experiment conditions defined in this proposal.

**Baseline definition for RQ3:** In the existing workflow, a TrekLink device triggers an SOS broadcast across the LoRa mesh. If a guide's Meshtastic-paired phone receives the alert, the guide contacts staff by voice call or messaging application. There is no structured acknowledgment, no timestamp recording, no response-action log, and no audit trail. Baseline MTTA is the elapsed time from SOS device trigger to the first documented verbal or text acknowledgment, recorded by an independent observer. Baseline traceability score is 0% by definition (no audit record exists). A reduction in MTTA is not assumed in advance; outcomes will be reported as measured.

**b. Research Objectives**

- Design and implement a priority-aware, offline-resilient Gateway Bridge with end-to-end idempotent event processing connecting the TrekLink LoRa mesh to a cloud backend.
- Evaluate gateway synchronization reliability under a controlled matrix of connectivity-loss durations and reconnection patterns, using physical TrekLink hardware for all mesh-to-gateway measurements.
- Design and implement an SOS-to-Incident pipeline with a formally defined FSM, tiered notification and escalation, and append-only audit trail.
- Evaluate incident-response workflow efficiency using MTTA, MTTR, traceability score, and incident-completion rate in randomized controlled emergency drills, compared with the uncoordinated baseline.
- Integrate the evaluated gateway and incident mechanisms into a complete multi-organization rental platform covering asset management, rental contracts, live telemetry, and billing.

**c. Research Scope & Methodology**

In scope: TrekLink ESP32/ESP32-S3 devices and LoRa mesh (Summer 2026 firmware with targeted enhancements); Field Station: LoRa-to-MQTT-to-cloud synchronization with priority queue and idempotency; SOS, GPS, and selected telemetry message types; connectivity-loss/recovery experiments in a controlled lab environment; SOS-to-Incident workflow evaluation using controlled drills; organizations, asset and rental-contract modules with multi-tenancy; responsive web client (desktop/tablet); sandbox payment only.

Out of scope: Mesh-stack rearchitecture, native mobile apps, commercial payment, hardware radio certification, individual renters, and the organizations' own trip and rescue operations.

Methodology:

- **Literature Review:** Survey LoRa mesh systems, store-and-forward and DTN protocols, IoT fleet management, and emergency incident workflows. Identify and articulate the integration gap.
- **Architecture & Design:** Freeze firmware message schema and eventId structure. Design gateway synchronization architecture, SQLite priority queue schema,

idempotency table, incident FSM, and experiment protocol. Produce SRS, UML State Machine Diagrams, architecture diagrams, and sequence diagrams.

- **Implementation:** Develop gateway bridge, backend modules, and web client per TP1–TP5.
- **Experiment 1: Gateway Synchronization (RQ1 & RQ2):** Use 3–5 physical TrekLink devices for all mesh-to-gateway reliability measurements. MQTT simulation scripts may supplement Gateway-to-Cloud workload evaluation only, not used to claim LoRa RF reliability. Apply connectivity-loss matrix: 0 s (control), 30 s, 2 min, 5 min, 10 min, 30 min; two reconnection patterns (immediate and delayed 60 s). Minimum 20 trials per condition. Measures: delivery rate (%), data-loss rate (%), duplicate rate (%), sync latency (mean / median / P95 in seconds), queue-recovery time (s), priority-ordering compliance rate (%).
- **Experiment 2: SOS-to-Incident Workflow (RQ3):** Controlled emergency drills with two organizations at once, each with 1 TrekLink device, 1 Field Station, an on-duty member and a backup, minimum 4 participants rotating. Scenario order randomized; participants do not know exact SOS trigger time; independent observer triggers SOS and starts external timer. All TrekLink-condition timestamps recorded automatically by the backend; baseline timestamps recorded by independent observer (hardware stopwatch). 15–20 drills per condition; vary network latency (LAN and 4G hotspot). Measures: MTTA (mean / median / P95 in seconds), MTTR (mean / median in minutes), traceability score (% of FSM transitions with complete actor + timestamp + action), missed-alert rate (%), duplicate-alert rate (%), incident-completion rate (%), cross-organization leakage (alerts or positions delivered to the wrong organization, expected 0).
- **Integration & System Testing:** End-to-end test cases for all functional requirements, idempotency tests (10× same eventId → 1 Incident), concurrency tests (20 simultaneous events → 0 duplicates), RBAC validation, audit-log completeness check, a cross-organization isolation suite with at least two organizations per module (including an event that synchronizes after its device was returned and rented to another organization), and a load test with simulated devices and Field Stations at 10 organizations, 50 rented devices and 20 simultaneous Field Station submissions.

**d. Expected Scientific Contribution**

- **A priority-aware, offline-resilient gateway pattern** for connecting intermittently connected LoRa mesh devices to cloud-based operational systems, with a formally defined eventId scheme, SQLite persistence model, and idempotent backend processing.
- **Empirical benchmark** of gateway synchronization reliability and latency under a controlled connectivity-loss matrix on low-cost ESP32 hardware.
- **An SOS-to-Incident workflow design** with a formally specified FSM, tiered notification and escalation chain, and append-only audit trail, directly traceable to measurable research outcomes.
- **Comparative evaluation** of uncoordinated vs. structured incident response (MTTA, MTTR, traceability) using randomized controlled drills.

- **A practical case study** demonstrating end-to-end integration of an open-source LoRa mesh prototype into a multi-organization rental platform for trekking agencies.

**e. Related Works / Literature Review (Preliminary)**

- **LoRa mesh communication** (Meshtastic, RAK WisBlock Mesh): Multi-hop messaging, GPS, local SOS broadcast, fall detection. Designed to work independently of any infrastructure; TrekLink builds on it and adds the organization-side layer: device rental and fleet management, store-and-forward to the cloud, and an incident pipeline.
- **IoT fleet management platforms** (AWS IoT Core, Azure IoT Hub, Balena): Device registration, telemetry ingestion, remote monitoring, lifecycle states. Gap: generic cloud platforms; no LoRa mesh integration; no store-and-forward for intermittent connectivity; no rental domain.
- **Delay/Disruption-Tolerant and store-and-forward systems** (DTN RFC 4838 Bundle Protocol, MQTT-SN, LoRaWAN ADR): Message persistence and forwarding under intermittent connectivity. Gap: protocol-level designs; not integrated with business workflows or incident management.
- **Incident management systems** (JIRA Service Management, PagerDuty, Opsgenie): Ticket lifecycle, escalation, notification, SLA tracking, audit trail. Gap: designed for IT/DevOps; no IoT field-event ingestion; no outdoor or trekking domain context.

Gap statement: TrekLink combines five dimensions in one domain-specific platform: off-grid LoRa mesh communication, store-and-forward synchronization under intermittent connectivity, device rental lifecycle management, real-time field monitoring, and structured SOS-to-incident response.

Key References:


- [1] A. W.-L. Wong, S. L. Goh, M. K. Hasan, and S. Fattah, “Multi-hop and mesh for LoRa networks: Recent advancements, issues, and recommended applications,” ACM Comput. Surv., vol. 56, no. 6, Article 136, pp. 1–43, Jan. 2024. DOI: 10.1145/3638241
- [2] Muladi, H. Wijaya, S. D. Prasetyo, S. A. Hamzah, and A. K. Mahamad, “LoRa mesh-based IoT GPS tracking system for mountain climbers,” Int. J. Safety Security Eng., vol. 14, no. 6, pp. 1751–1761, Dec. 2024. DOI: 10.18280/ijsse.140610
- [3] H. Zhang, R. Zhang, and J. Sun, “Developing real-time IoT-based public safety alert and emergency response systems,” Sci. Rep., vol. 15, Article 29056, Aug. 2025. DOI: 10.1038/s41598-025-13465-7
- [4] OASIS, “MQTT Version 5.0,” OASIS Standard, Mar. 2019. https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html
- [5] V. Cerf, S. Burleigh, A. Hooke, L. Torgerson, R. Durst, K. Scott, K. Fall, and H. Weiss, “Delay-tolerant networking architecture,” IETF RFC 4838, Apr. 2007. https://www.rfc-editor.org/rfc/rfc4838
- [6] Meshtastic Project, “Official Documentation,” 2024. [Online]. Available: https://meshtastic.org/docs/


**4. Other comments (propose all relative things if have):**

The TrekLink LoRa mesh firmware and hardware (ESP32/ESP32-S3) are an output of this team's Summer 2026 project and the technical foundation of this Capstone. Targeted firmware enhancements made during the Capstone are in scope and credited; the hardware design effort is documented in the reports.

| | |
|---|---|
| **Supervisor (If have)***(Sign and full name)* | **HCM, date ____/____/2026On behalf of Registers***(Sign and full name)* |